Autor: Ing. Eduardo Avilés

Fecha: 14/03/2026

1. Objetivo:

Presentar la solución ante el problema planteado del reto técnico Devsu.

2. Requerimientos del reto:

2.1. Gestión financiera (lógica de negocio)

- Consultas de movimientos bancarios y detalles de productos financieros
- Datos al detalle del cliente obtenidos mediante un sistema independiente
- Realización de transferencias entre cuentas del banco e interbancarias
- Registro de nuevos clientes con reconocimiento facial en la app móvil
- Sistema de notificaciones y alertas en movimientos mínimo 2 entre propios y externos

2.2. Aplicaciones Frontend

- Web simple page aplication acceso desde navegadores
- App móvil con framework multiplataforma
- Autenticación mediante huella, reconocimiento facial u otro después del onboarding inicial

2.3. Integración y arquitecturas

- Uso de OAUTH2.0 en autenticación, se integra mediante un producto existente del banco
- Implementación de api Gateway para centralizar el consumo de servicios
- Microservicios para consulta de datos básicos, consulta de movimientos, transferencias.
- Base de datos de auditoría para registro de todas las acciones
- Implementación de un mecanismo de persistencia optimizado para los clientes frecuentes

2.4. Atributos de calidad de software y cumplimiento de normativas

- Infraestructura orientada a la nube de AWS que garantice baja latencia
- Alta disponibilidad, tolerancia a fallos y recuperación ante desastres
- Excelencia en operaciones, monitoreo constante con capacidad de auto-healing
- Cumplimiento de leyes de protección de datos, gestión de seguridad de la información (LODP en Ecuador y RGDP en estándares internacionales, iso 27001 y 27701) estándares de seguridad financiera

3. Arquitectura general del sistema

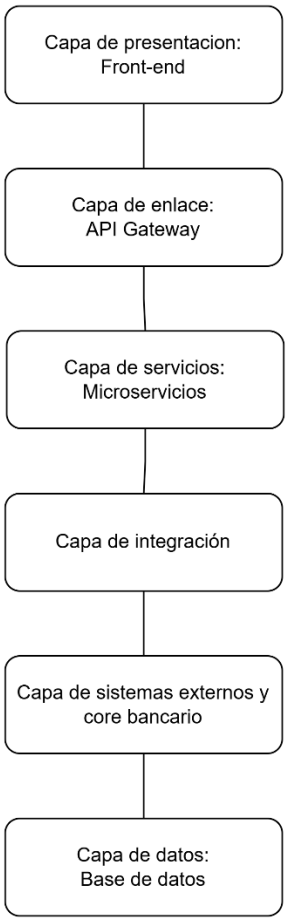
La arquitectura que presento como solución es: Microservicios + Envent Driven + Cloud Native. Esta arquitectura permite cumplir con los requerimientos de consulta de movimientos, notificaciones, auditoría y sobre todo garantiza una alta disponibilidad.

Al optar por microservicios garantizo bajo acoplamiento ya que separo la consulta de transacciones con las transferencias, haciendo cada proceso independiente.

Event driven es ideal ya que las acciones propias del cliente, como ejemplo transferencia de dinero, representan un evento, mediante el uso de un bróker como Kafka o SNS/SQS de AWS puedo solventar el uso de 2 sistemas de notificación y garantizo con esto baja latencia.

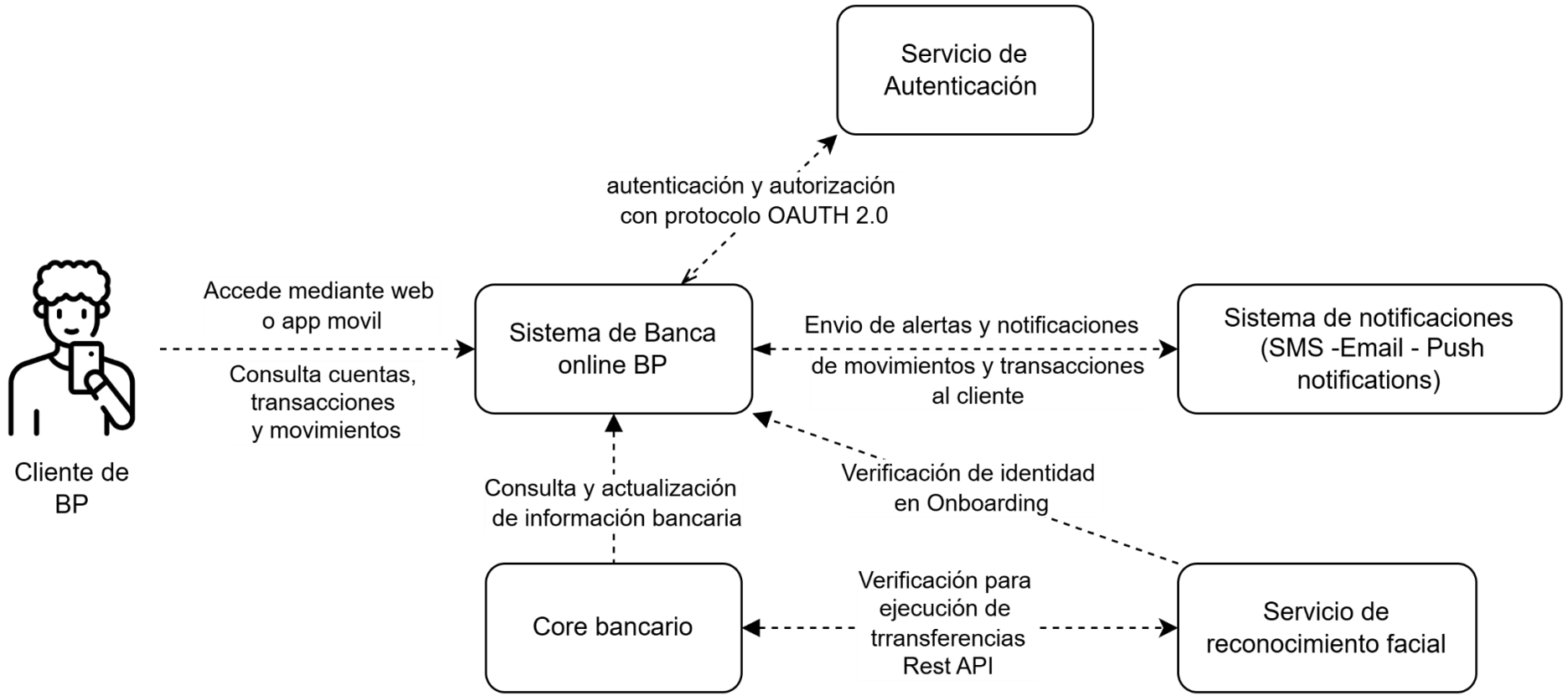
Por otro lado, Cloud native permite hacer un sistema resiliente y que cumpla con el requisito de auto-healing aprovechando capacidades nativas del cloud computing como contenedores, serveless y managed services. Netamente el uso de cloudwatch me conviene para el monitoreo y observabilidad, mientras que EKS como orquestador permite el auto-healing.

Para representar la arquitectura en nivel conceptual mediante capas se tiene la siguiente figura:

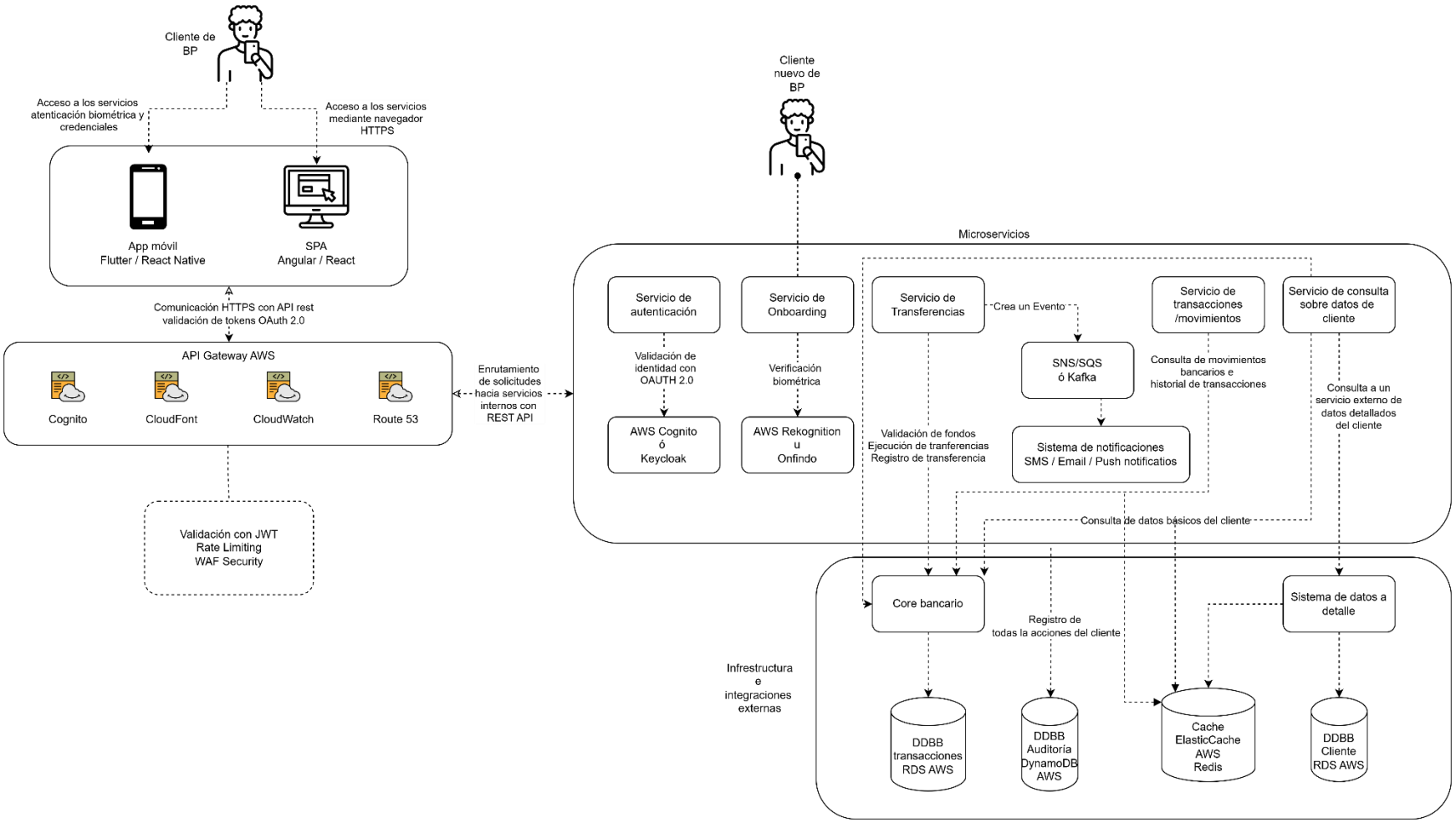


4. Diagrama C4

4.1 Diagrama de contexto del sistema



4.2 Diagrama de contenedores

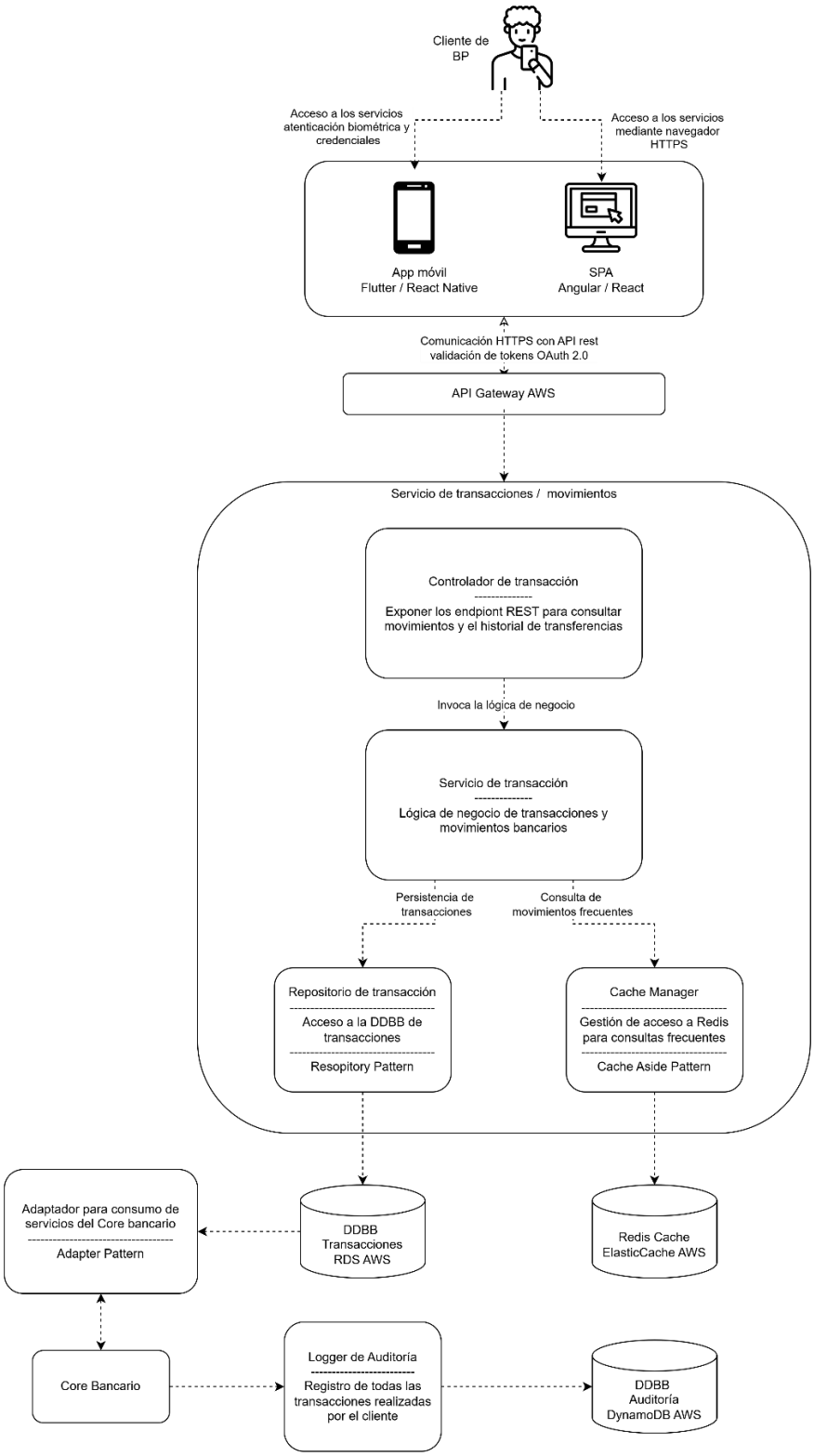


4.3 Diagrama de componentes

Contenedor: Servicio de transacciones / movimientos

Alternativas para desarrollo: Java Spring Boot o NodeJs

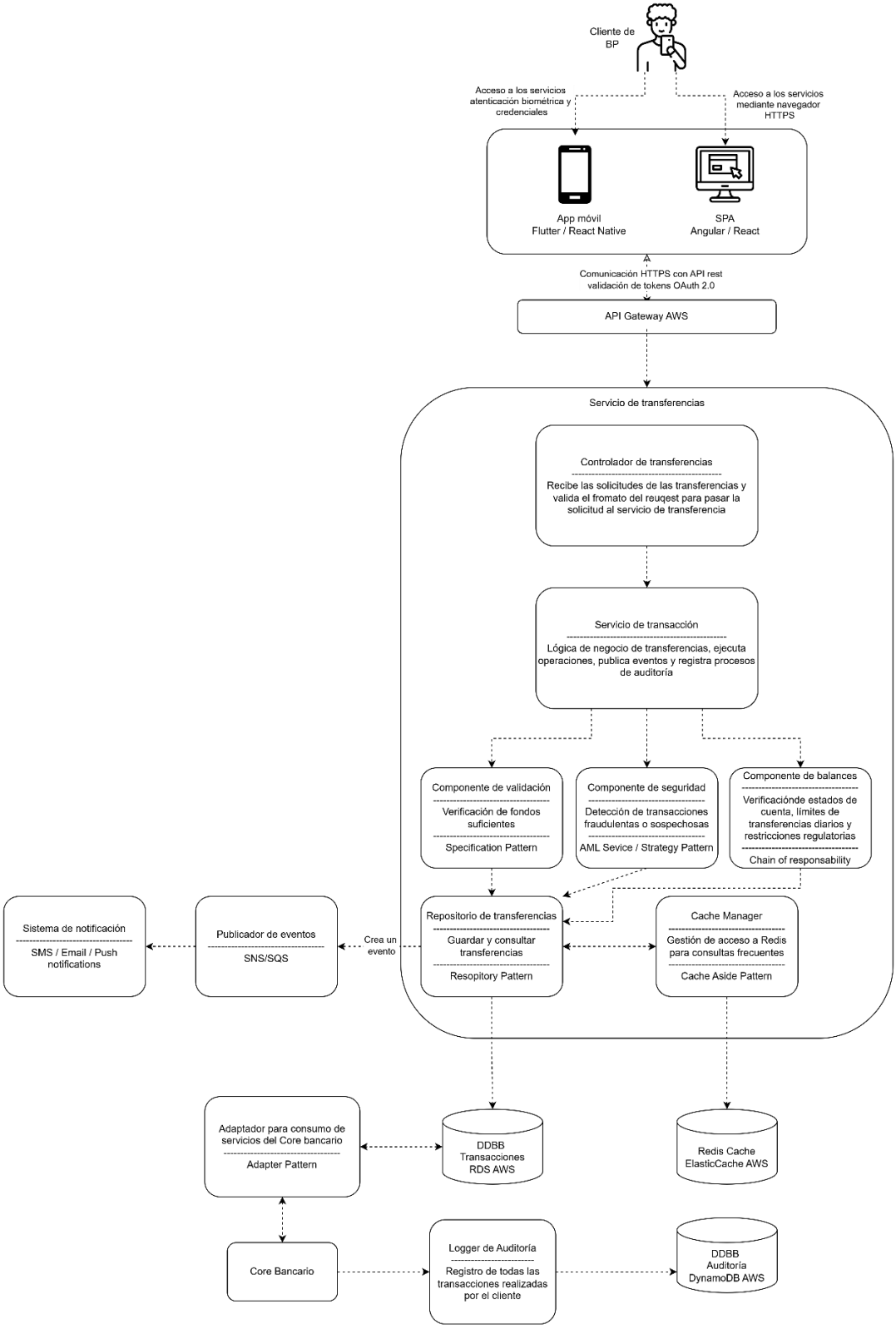
Seguridad: Cifrado en tránsito mediante TLS y credenciales manejadas con AWS Secrets Manager



Contenedor: Servicio de transferencias

Alternativas para desarrollo: Java Spring Boot o NodeJs

Seguridad: Cifrado en tránsito mediante TLS y credenciales manejadas con AWS Secrets Manager, autenticación y autorización previa con AWS Cognito, AWS WAF, IAM y cifrado en reposo



5. Justificaciones

5.1. Arquitectura general

La arquitectura propuesta se basa en el enfoque de Clean Architecture separando por capas los componentes y siguiendo principios de alta cohesión y bajo acoplamiento, logrando así que cada componente del sistema tenga sus responsabilidades claras y bien definidas.

Esto facilita el mantenimiento, la escalabilidad, evolución y resiliencia del sistema y la reutilización de componentes. Si bien es cierto que le acoplamiento dentro de una arquitectura no se va a eliminar completamente, esta arquitectura busca minimizarlo y controlarlo con el uso de APIs bien definidas y separando responsabilidades.

5.2 Front-end

Ya que la aplicación contempla dos aplicaciones: web y móvil, se recomienda utilizar las tecnologías de:

Angular para la aplicación web: Es un framework robusto ideal para desarrollo de aplicaciones dinámicas con complejidad alta y muy demandante, la arquitectura que se basa en componentes facilita el mantenimiento del código y posee alto soporte y escalabilidad.

React Native para la aplicación web y móvil: Ya que React native permite el desarrollo de aplicaciones multiplataforma, se puede desarrollar software con un solo código base tanto para web como para móvil, este framework permite tener un amplio ecosistema de librerías, alto rendimiento y escalabilidad muy cercano a aplicaciones nativasy facilita la integración con librerías de biometría para autenticación. Un apartado de consideración es que React native cuenta con Expo para el entorno de compilación en tiempo real lo que facilita y agiliza el desarrollo, reduciendo costos y mantenimiento sin sacrificar experiencia de usuario.

Flutter para aplicación móvil: Ofrece un gran rendimiento garantizando la identidad visual de la aplicación, ofrece fluidez en los screens y es un framewok robusto y seguro, punto a favor es que la velocidad de despliegue de parches de seguridad es mayor ante una vulnerabilidad y también tiene integración nativa con servicios de reconocimiento facial y biometría, en cuanto a costo representa un costo operativo muy reducido.

5.3 Tecnologías de backend

Se propone el uso de **NodeJs** por la experiencia en mercado ya que lleva años y es muy confiable por su modelo de ejecución asíncrono, ideal para el alto número de solicitudes concurrentes de una banca virtual.

Por otro lado, se propone como otra alternativa a **Spring Boot** ya que es muy utilizado en sistemas empresariales y financieros debido a la robustez que presenta. Se podría manejar las transacciones de forma sólida y tiene un amplio soporte de seguridad y validaciones, se puede optar por la integración con patrones de resiliencia como circuit braker.

5.4 Microservicios y mensajería

La arquitectura de microservicios permite dividir el sistema en componentes independientes alineados con el giro de negocio lo que se traduce prácticamente en

escalabilidad independiente (horizontal), despliegue independiente, resiliencia y facilidad de evolución.

Por otra parte, para la comunicación entre los servicios se propone el uso de eventos y mensajería asíncrona mediante servicios de nube de Amazon como SNS y SQS. Como alternativa se puede optar por el uso de Kafka. Esto permite desacoplar servicios y mejorar totalmente la resiliencia del sistema.

5.5 Seguridad y cumplimiento normativo

Ya que el sistema debe cumplir con normativas de protección de datos personales y de seguridad financiera se debe tener en cuenta las siguientes regulaciones y leyes:

Ley orgánica de protección de datos personales (LODP): esta ley permite establecer obligaciones en protección de datos personales, tratamiento seguro de la información, consentimiento del usuario y la confidencialidad de datos. En ese sentido el sistema implementa controles de seguridad como cifrado de datos, autenticación robusta y la auditoría de accesos.

Reglamento general de protección de datos (GRDP): En el caso de que el sistema realice transacciones internacionales o la interacción con clientes de otros países es necesario considerar este reglamento con los lineamientos necesarios en consideración a LOPD.

5.6 Protocolos de seguridad

Oauth 2.0 es el estándar para autenticación y autorización de usuarios que permite delegar la autenticación a un proveedor seguro, proteger mediante tokens las APIs e implementar una autenticación segura para las aplicaciones clientes (web y móvil).

TLS: todas las comunicaciones del sistema deben usar TLS para garantizar confidencialidad de la información, integridad de datos y protección contra interceptación de comunicaciones.

AML (anti-Money Laundering): hoy en día y por la realidad que existe en el país en necesario implementar mecanismos de prevención contra lavado de dinero, AML permite analizar transacciones sospechosas, verificar limites regulatorios mandatarios y generar alertas ante posibles fraudes.

5.7 Flujo de autenticación

Se recomienda utilizar el flujo **Authorization Code Flow con PKCE** de Oauth 2.0 con el objetico de evitar la exposición de credenciales de usuario, proteger contra ataques de intercepto de código, emitir tokens de acceso seguros para consumir APIs.